MINI DISTRIBUTED CLOUD PLATFORM

Complete Project Vision · Feature Scope · Production Readiness · Weekly Deliverables

4	4	4	2
Laptops as nodes	Core AWS services simulated	Weeks build timeline	Engineers parallel tracks

CONTENTS

1. What We Are Building — The Big Picture.....Pg 3
2. System Architecture — How It All Connects.....Pg 4
3. Feature 1 — Compute Service (EC2-like).....Pg 5
4. Feature 2 — Storage Service (S3-like).....Pg 6
5. Feature 3 — Function Execution (Lambda-like).....Pg 7
6. Feature 4 — Monitoring & Logging.....Pg 8
7. In Scope vs Out of Scope.....Pg 9
8. Production Readiness — What It Takes.....Pg 10
9. Weekly Deliverables — What Gets Done When.....Pg 12
10. Success Criteria — How We Know It Works.....Pg 16

1 · What We Are Building — The Big Picture

A working mini cloud platform across 4 laptops that simulates EC2, S3, Lambda and CloudWatch

The Core Idea

Imagine you walked into AWS's data centre and saw only 4 laptops. One laptop acts as the control centre (master) — it receives requests, makes decisions, and coordinates everything. Three laptops are the workers — they actually run containers, store files, and execute code. Together they behave like a real cloud provider: accepting API calls, running workloads, storing data, and reporting their health.

The key insight: AWS is not magic. It is a collection of well-engineered software components talking to each other over a network. This project builds those same components from scratch in Python, which means by the end you will not just know how to use AWS — you will understand how it works internally.

What Problem Does Each Service Solve?

AWS Service	What You're Simulating	The Problem It Solves	Real-World Analogy
EC2	Compute Service	Run application containers across multiple machines without SSH-ing into each one manually	A restaurant chain's kitchen — orders come to the front desk, manager assigns them to the right kitchen (worker)
S3	Storage Service	Store and retrieve files reliably even when individual machines fail	A library with multiple branches — if one branch burns down, the books are still available at another
Lambda	Function Execution	Run a piece of code on demand without provisioning a server yourself	A vending machine — you press a button, get your item, the machine doesn't stay on for you between uses
CloudWatch	Monitoring & Logging	Know what every machine is doing at any moment, and alert when something goes wrong	A hospital with a central monitoring station showing all patients' vitals simultaneously

What Makes This Different From Just Running Docker?

Docker on a single machine gives you isolation. This project adds the distributed layer on top:

- A scheduler that decides WHICH machine runs your container based on real-time CPU and RAM
- A storage system that copies your files to multiple machines so they survive hardware failure
- A heartbeat system that detects when a machine dies and automatically compensates
- A monitoring layer that gives you a single dashboard across all 4 machines simultaneously
- An API gateway so users interact with one endpoint and never need to know which machine handled their request

2 · System Architecture — How It All Connects

Two-tier hierarchy: one master node coordinates, three worker nodes execute

Node Roles

Node	Machine	IP	Runs	Does NOT Run
Master	Laptop 1	192.168.1.10	FastAPI API gateway, Scheduler, Node Registry, Storage Manager, PostgreSQL, Redis	Docker containers for user tasks, File chunks
Worker 1	Laptop 2	192.168.1.11	FastAPI worker API, Docker Engine, Task Executor, Chunk Storage, Function Runner, Health Reporter	Scheduling logic, User-facing API
Worker 2	Laptop 3	192.168.1.12	Same as Worker 1	Same as Worker 1
Worker 3	Laptop 4	192.168.1.13	Same as Worker 1	Same as Worker 1

How a Request Flows Through the System

Every user interaction follows the same fundamental path. Understanding this flow means understanding the entire architecture.

Step	What Happens	Which Component	Network Call?
1	User sends HTTP request to master (e.g. POST /tasks)	API Gateway (FastAPI)	Incoming — user to master
2	Master validates request using Pydantic models	Shared models (shared/models.py)	None — in-memory
3	For compute tasks: scheduler scores all healthy workers	Task Scheduler	None — reads heartbeat registry
4	Master sends task to the chosen worker	Master → Worker REST call	Yes — WiFi LAN
5	Worker runs the task in a Docker container	Task Executor (docker-py)	None — local Docker socket
6	Worker sends result back to master	Worker → Master callback	Yes — WiFi LAN
7	Master stores result in PostgreSQL, responds to user	Task Repository	None — local DB
8	Worker sends updated CPU/RAM stats in next heartbeat	Health Reporter	Yes — every 5 seconds

Communication Rules

- Workers NEVER talk directly to each other. All coordination goes through master.
- Master NEVER executes user workloads. It only coordinates.
- All inter-node communication is REST over HTTP — readable, debuggable, inspectable with curl.
- All shared data contracts are Pydantic models in shared/models.py — the single source of truth.
- Master exposes port 8000. Workers expose port 8001. These never change.

3 · Feature 1 — Compute Service (EC2-like)

Deploy and run containerised workloads across worker nodes

What It Does

The compute service lets a user submit a task — a Docker image name and a command — and receive the output back without knowing or caring which physical machine ran it. The master handles scheduling, the worker handles execution.

User Experience (what the API feels like):

API Contract — Compute

Submit task: POST /tasks body: { "image": "python:3.11-slim", "command": "python3 -c \"print(2+2)\"", "cpu_limit": 0.5, "ram_limit_mb": 256 }

Response: { "task_id": "abc-123", "status": "accepted" }

Poll result: GET /tasks/abc-123

Result: { "task_id": "abc-123", "status": "COMPLETED", "output": "4\n", "worker_id": "worker-2", "duration_seconds": 1.2 }

Scheduling — The Core Algorithm

Every time a task arrives, the scheduler scores every healthy worker and sends the task to the highest-scoring one:

Scoring Formula

score = (1 - cpu_percent/100) × 0.5 + (1 - ram_percent/100) × 0.3 + (1 - active_tasks/MAX) × 0.2 Higher score = more available = better candidate. Raise NoHealthyNodeError if all nodes are dead.

Scheduling Concept	Why It Matters	Example
Resource-aware scoring	Your 4 laptops likely have different specs. Round-robin would overload the weakest one.	Laptop A has 32GB RAM, Laptop B has 8GB. Laptop A gets a higher score and more tasks.
Active task weighting	CPU% alone doesn't tell you the queue depth. A node could be idle but about to get 5 tasks.	Worker with 3 active tasks scores lower than one with 0, even if CPU% looks similar.
Concurrency limit	Each worker has asyncio.Semaphore(3) — max 3 tasks at once. Prevents overwhelming a node.	If worker-1 is full, master gets a 429 and automatically tries the next best worker.
Task reaper	If a worker dies mid-task, the task stays RUNNING forever without a reaper.	Background loop every 30s: find tasks RUNNING > 10 min on dead nodes → mark FAILED → retry.

Docker Execution — What Actually Runs on the Worker

Docker Parameter	Value	Why
image	User-specified (e.g. alpine)	Defines the environment the task runs in
command	User-specified (e.g. echo hello)	What executes inside the container
detach	False	Wait for completion before returning
remove	True	Auto-delete container after finish — no cleanup needed
mem_limit	f'{ram_limit_mb}m'	Hard cap on memory — prevents OOM on the host
nano_cpus	int(cpu_limit × 1e9)	Hard cap on CPU cores (0.5 = half a core)
network_mode	'none'	Security — user code cannot make external network calls
stdout/stderr	True	Capture all output to return as task result

4 · Feature 2 — Storage Service (S3-like)

Distributed file storage with replication and failure recovery

What It Does

Users upload files to the master via API. The master splits them into 1MB chunks, stores each chunk on 2 different workers, and records the chunk locations in PostgreSQL. If a worker dies, the file is still accessible from the replica. A background monitor detects under-replication and fixes it automatically.

User Experience:

API Contract — Storage

Upload: POST /files body: { "filename": "data.csv", "content": "<base64-encoded bytes>" }

Response: { "file_id": "xyz-456", "size_bytes": 5242880, "chunks": 5, "replicated_on": ["worker-1", "worker-2"] }

Download: GET /files/xyz-456 → returns base64 content

List: GET /files → returns all file metadata

How Files Are Stored — Step by Step

Step	What Happens	Production Consideration
1 — Receive	User POSTs file to master. Master decodes base64 bytes.	Validate file size limit (configurable, default 100MB). Reject early if too large.
2 — Chunk	File split into 1MB chunks: bytes[0:1048576], bytes[1048576:2097152], etc.	Record chunk_index so download can reassemble in correct order.
3 — Pick workers	For each chunk, pick 2 healthy workers that are different from each other.	Ensure replicas are on different physical machines — no point replicating to the same node.
4 — Send chunks	POST each chunk to both chosen workers via /chunks endpoint. Workers store in /tmp/minicloud-storage/{node_id}/.	Verify MD5 checksum of the stored chunk matches the sent data. Reject corrupt chunks.
5 — Record	Save FileMetadata and FileChunk records in PostgreSQL on master.	FileChunk records: {chunk_id, file_id, chunk_index, checksum, worker_id}. Two records per chunk.
6 — Confirm	Return file_id to user.	If any chunk fails to replicate, roll back — return error, don't leave partial uploads.

Failure Recovery — Replication Monitor

A background task runs every 60 seconds on the master to ensure every chunk has its full replication factor maintained:

- Query PostgreSQL for FileChunks whose worker_id belongs to a node currently marked 'dead'
- Find the surviving replica of that chunk on an alive worker
- GET the chunk bytes from the surviving worker
- POST them to a new healthy worker to restore replication factor to 2

- Update the FileChunk record in PostgreSQL with the new worker_id
- Mark FileMetadata.status = 'degraded' during re-replication, 'available' once complete

Checksum Integrity

Every chunk has an MD5 checksum recorded at upload time. On download, we recompute the MD5 of the retrieved bytes and compare. A mismatch means storage corruption — we immediately try the replica. If both copies are corrupt, we return an error rather than silently serving bad data. Silent data corruption is one of the hardest bugs to debug in a distributed system.

5 · Feature 3 — Function Execution (Lambda-like)

Run user-submitted Python functions on demand in isolated containers

What It Does

A user uploads a Python function as a string. Later, they invoke it with a JSON payload. The master schedules the invocation to a worker. The worker writes the code to a temp file, injects the payload, mounts it into a fresh Docker container, runs it, captures the output, and returns it. The container is destroyed immediately after.

User Experience:

API Contract — Functions

Upload: POST /functions body: { "name": "add_numbers", "code": "import json\nresult = PAYLOAD[\"x\"] + PAYLOAD[\"y\"]\nprint(json.dumps({'result': result}))" }

Response: { "function_id": "fn-789" }

Invoke: POST /functions/fn-789/invoke body: { "payload": {"x": 5, "y": 3}, "timeout_seconds": 30 }

Result: { "output": "{\"result\": 8}\n", "error": "", "duration_ms": 340, "worker_id": "worker-3" }

Execution Model — What Happens on the Worker

Step	Detail	Security / Production Note
1 — Receive code	Worker receives FunctionDef (function_id, name, code string) + InvokeRequest (payload dict)	Code arrives as a plain string — never eval() it directly outside a container.
2 — Write temp file	Write code to /tmp/fn_{invocation_id}.py. Prepend: PAYLOAD = {json.dumps(payload)}	Unique invocation ID per call prevents concurrent invocations from overwriting each other's file.
3 — Docker run	docker run --rm -v /tmp/fn_{id}.py:/app/handler.py --memory=128m --cpus=0.25 --network=none python:3.11-slim python /app/handler.py	--network=none: user code cannot make HTTP calls or exfiltrate data. --rm: container auto-deletes.
4 — Capture output	stdout → output field. stderr → error field. Exit code != 0 → status = FAILED.	Truncate output at 1MB to prevent memory exhaustion from malicious print loops.
5 — Timeout	asyncio.wait_for(run_container(), timeout=30). On timeout: container.kill(), return FAILED.	Without timeout, a function that sleeps forever would hold the semaphore slot indefinitely.
6 — Cleanup	Delete /tmp/fn_{id}.py after completion whether success or failure.	Use try/finally to guarantee cleanup even if an exception occurs.

Key Concepts to Understand

Concept	What It Is	How We Handle It

Concept	What It Is	How We Handle It
Cold start	First invocation pulls Docker image — can take 10–30 seconds.	Pre-pull python:3.11-slim on worker startup. First invoke is slow but subsequent ones are fast.
Stateless execution	Function cannot access data from previous invocations.	By design — each run gets a fresh container. Use the storage service if persistence is needed.
Payload injection	User code needs its input data. We inject it as a Python variable.	Prepend PAYLOAD = {...} to the file. User code reads PAYLOAD directly.
Concurrent invocations	Multiple users invoke the same function simultaneously.	Each invocation gets a unique temp file path using a UUID. Semaphore limits max concurrent executions.

6 · Feature 4 — Monitoring & Logging (CloudWatch-like)

Real-time visibility into all 4 nodes from a single dashboard

What It Does

Every worker exposes a /metrics endpoint in Prometheus format. A Prometheus server running on the master scrapes all workers every 15 seconds. Grafana reads from Prometheus and renders dashboards. Simultaneously, all structured logs flow from workers to master and are stored in PostgreSQL, queryable by task_id, node, or severity.

Metrics Collected

Metric Name	Type	Node	What It Tracks
minicloud_healthy_nodes	Gauge	Master	Count of workers currently in 'healthy' status
minicloud_tasks_submitted_total	Counter	Master	Cumulative tasks submitted since startup
minicloud_tasks_by_status	Gauge	Master	Current count of tasks in each status (PENDING, RUNNING, etc.)
minicloud_files_stored	Gauge	Master	Total number of files in the storage system
minicloud_worker_cpu_percent	Gauge	Each Worker	Current CPU usage % (from psutil)
minicloud_worker_ram_percent	Gauge	Each Worker	Current RAM usage % (from psutil)
minicloud_worker_active_tasks	Gauge	Each Worker	Tasks currently executing on this worker
minicloud_worker_tasks_completed_total	Counter	Each Worker	Cumulative completed tasks on this worker
minicloud_worker_tasks_failed_total	Counter	Each Worker	Cumulative failed tasks on this worker
minicloud_worker_storage_used_bytes	Gauge	Each Worker	Bytes of chunk data stored locally

Structured Logging Format

Every log line from every node is a JSON object. This makes logs filterable, searchable, and parseable by machines — not just humans:

Log Line Example

{ "timestamp": "2024-01-15T10:23:45.123Z", "level": "INFO", "node_id": "worker-2", "component": "task_executor", "event": "task_completed", "task_id": "abc-123", "duration_seconds": 2.4, "image": "alpine", "message": "Task completed successfully" }

Log Field	Purpose	Example Value
timestamp	ISO 8601 — enables time-range queries	2024-01-15T10:23:45.123Z
level	Severity — filter by ERROR to find problems quickly	INFO | WARNING | ERROR | CRITICAL
node_id	Which machine emitted this log	worker-2, master

Log Field	Purpose	Example Value
component	Which subsystem inside the node	task_executor, heartbeat, scheduler
event	Machine-readable event name — enables counting/alerting	task_completed, node_marked_dead, chunk_replicated
task_id / file_id	Correlation ID — trace a full request across master and worker logs	abc-123
message	Human-readable description	Task completed successfully

Grafana Dashboards

Dashboard	Panels	Answers the Question
Cluster Overview	Healthy node count, tasks submitted today, tasks by status (pie), active tasks right now, total files stored	Is the cluster healthy? Is work getting done?
Worker Nodes	CPU% per worker (time series, all on one chart), RAM% per worker, active tasks per worker, storage used per worker	Which worker is the bottleneck? Which is underutilised?
Task History	Completion rate over time, failure rate, average duration, P95 duration	Is performance degrading? Are failures increasing?
Storage Health	Replication events, chunks under-replicated, re-replication events	Is data safe? Are any files at risk?

7 · In Scope vs Out of Scope

What this project builds, and what it deliberately does not

Explicitly In Scope

Feature	Description	Done In
Node registration	Workers self-register with master on startup via POST /nodes/register	Week 1
Heartbeat monitoring	Workers POST health data every 5s; master detects dead nodes within 15-20s	Week 1
Redis registry persistence	Node registry survives master restarts via Redis backup	Week 1
Structured logging	JSON-format logs from all nodes centralised to master PostgreSQL	Week 1
Task submission API	POST /tasks accepts Docker image + command, returns task_id	Week 2
Resource-aware scheduling	Scoring formula selects least-loaded worker per task	Week 2
Docker container execution	Worker runs tasks in isolated containers with CPU/RAM limits	Week 2
Task state machine	PENDING → SCHEDULED → RUNNING → COMPLETED/FAILED lifecycle	Week 2
Concurrency control	asyncio.Semaphore limits 3 concurrent tasks per worker	Week 2
Task reaper	Background loop reschedules orphan tasks from dead workers	Week 2
Retry logic	Failed tasks retry up to 3 times on different workers	Week 2
Chunked file upload	Files split into 1MB chunks and distributed across workers	Week 3
Replication factor 2	Each chunk stored on 2 different workers	Week 3
Checksum verification	MD5 per chunk verified on upload and download	Week 3
Replication monitor	Background task restores replication after worker failure	Week 3
Lambda function upload	Users upload Python code as string, stored in PostgreSQL	Week 3
Lambda invocation	Code injected into Docker container, output returned	Week 3
Prometheus metrics	All nodes expose /metrics; Prometheus scrapes every 15s	Week 4
Grafana dashboards	4 dashboards covering cluster health, workers, tasks, storage	Week 4
Log aggregation	Workers POST structured logs to master; queryable by task_id	Week 4
Stress testing	50 concurrent tasks, worker death mid-execution, replication failure	Week 4
Architecture documentation	Decision records explaining every major technical choice	Week 4

Explicitly Out of Scope (for this build)

Feature	Why Out of Scope	Could Be Added Later?
Authentication / API keys	Adds significant complexity with little learning value at this stage	Yes — add JWT auth in a Phase 2
HTTPS / TLS	Local LAN traffic over plain HTTP is acceptable for this project	Yes — nginx reverse proxy with self-signed certs
Multi-tenancy	Isolating one user's workloads from another's requires significant security work	Yes — add user accounts and resource quotas
Horizontal master scaling	Running multiple masters requires consensus (Raft/etcd) — out of scope	Yes — this is what Kubernetes does
Erasure coding	Splitting data mathematically for space efficiency is complex to implement	Yes — replace replication with erasure coding
Container image registry	Building a private Docker registry is a project in itself	Yes — add a local registry with registry:2
Network policies	Controlling which containers can talk to which is Kubernetes-level	Yes — add via Docker networking rules
Persistent container storage	Containers with volumes that survive restarts requires more orchestration	Yes — add Docker volumes in Phase 2
CLI client	A command-line tool for the API is a nice-to-have	Yes — build with Click or Typer in Phase 2
Kafka for messaging	Redis Streams is sufficient for this scale	Yes — replace Redis Streams with Kafka if scaling up

8 · Production Readiness — What It Takes

The difference between a demo that works once and a system that works reliably

What 'Production Ready' Means for This Project

Production ready does not mean it handles Netflix-scale traffic. It means: the system fails gracefully, recovers automatically, can be debugged when something goes wrong, and a new engineer can understand and run it without asking anyone. Here is every aspect addressed:

8.1 — Fault Tolerance

Failure Scenario	How the System Handles It	Code Component
Worker node dies	Heartbeat timeout → node marked dead. Task reaper detects orphaned tasks → reschedules to alive workers. Replication monitor detects under-replicated chunks → re-replicates.	health_monitor.py, task_reaper.py, replication_monitor.py
Master restarts	Redis stores full node registry. On startup, master reads Redis and immediately knows all alive workers — no re-registration needed.	node_registry.py with Redis backend
Task times out	asyncio.wait_for(timeout=300) kills the container. Task marked FAILED with error='timeout'. Retried on another worker if retry_count < max_retries.	docker_runner.py, task_executor.py
Worker disk full	Storage stats in heartbeat include disk_percent. Scheduler checks this and avoids sending chunks to workers above 90% disk usage.	local_storage.py, scheduler.py
Chunk corruption	MD5 checksum mismatch on download → try replica. If both corrupt → return 500 with clear error message, log the chunk IDs.	storage_manager.py
Worker at capacity	Semaphore full → worker returns 429. Master retries on next-best worker. If all workers full → return 503 to user with 'retry later' message.	task_executor.py, task_routes.py
Network partition (WiFi hiccup)	Heartbeat timeout set to 20s (4 missed beats) to tolerate momentary WiFi jitter. Reconnected workers automatically re-register.	health_monitor.py

8.2 — Observability

A system you cannot observe is a system you cannot debug. These three pillars must all work:

Pillar	What We Build	Why It Matters
Metrics	Prometheus scrapes /metrics from all nodes every 15s. Grafana visualises CPU, RAM, task counts, storage, error rates over time.	Tells you WHAT is happening: 'Worker 2 CPU spiked at 10:45am'. Without metrics you're guessing.
Logs	Structured JSON logs from all nodes, centralised to master PostgreSQL, queryable by task_id, node_id, level, time range.	Tells you WHY it happened: 'Task abc-123 failed because the Docker image alpine:badtag does not exist'. Without logs you cannot debug.

Pillar	What We Build	Why It Matters
Traces (simplified)	Every task and file operation has a unique ID that appears in every related log line and metric label.	Tells you the full STORY: trace task abc-123 from POST /tasks through scheduler decision through Docker run through result callback. Without this you lose cross-node context.

8.3 — API Design

Design Principle	How We Apply It	Example
Idempotent operations	Submitting the same task twice should not run it twice. Tasks have deterministic IDs.	POST /tasks with task_id='abc' twice → second call returns existing task, does not create duplicate
Clear error messages	Every error response includes: HTTP status code, machine-readable error code, human-readable message, and the resource ID that caused the error.	{ "error": "NO_HEALTHY_WORKERS", "message": "No healthy workers available. Try again in 30 seconds.", "status": 503 }
Consistent response shapes	All endpoints follow the same structure: { data: {...}, error: null } or { data: null, error: {...} }	Makes client code predictable — always check the same field
Pagination	GET /tasks returns max 50 results by default. Use GET /tasks?limit=100&offset=50 for pagination.	Prevents a single query from returning 10,000 tasks and OOM-ing the master
Async-first	POST /tasks returns immediately with task_id and status=PENDING. Client polls GET /tasks/{id} for result. Never block the API waiting for Docker.	Users get a response in milliseconds. Task may take minutes. Non-blocking is production behaviour.

8.4 — Code Quality Standards

Standard	What It Means	How We Enforce It
Type hints everywhere	Every function has typed arguments and return types: def select_worker(task: TaskRequest) -> tuple[NodeInfo, SchedulerDecision]	Catch bugs at development time, not runtime. Required for all new code.
Pydantic validation on all boundaries	Every piece of data that crosses a network boundary passes through a Pydantic model. No raw dicts.	shared/models.py is the contract. If a model field is missing or wrong type, Pydantic raises before the bug propagates.
No hardcoded values	IPs, ports, timeouts, chunk sizes, replication factors — all in a config file or environment variable.	infra/config.py with defaults. Override via environment variables for different environments.
Unit tests for all logic	Scheduler algorithm, storage chunking, heartbeat timeout detection — all have unit tests that run without any network or Docker.	tests/ folder. Run: python -m pytest tests/ -v before every PR.
Integration tests for all flows	Full end-to-end flows tested on actual hardware: task submission to completion, file upload to download after worker failure.	tests/test_week*_integration.py. Run weekly before merging to main.
Docstrings on public	Every public function has a one-line docstring	Code review — no undocumented public

Standard	What It Means	How We Enforce It
functions	explaining what it does and what it returns.	functions merged to dev.

8.5 — Configuration Management

infra/config.py — Central Config File

```
MASTER_HOST = os.getenv('MASTER_HOST', '192.168.1.10')
MASTER_PORT = int(os.getenv('MASTER_PORT', '8000'))
WORKER_PORT = int(os.getenv('WORKER_PORT', '8001'))
HEARTBEAT_INTERVAL_S = int(os.getenv('HEARTBEAT_INTERVAL_S', '5'))
HEARTBEAT_TIMEOUT_S = int(os.getenv('HEARTBEAT_TIMEOUT_S', '20'))  # 20s for WiFi, 15s for wired
MAX_CONCURRENT_TASKS = int(os.getenv('MAX_CONCURRENT_TASKS', '3'))
CHUNK_SIZE_BYTES = int(os.getenv('CHUNK_SIZE_BYTES', str(1024*1024)))  # 1MB
REPLICATION_FACTOR = int(os.getenv('REPLICATION_FACTOR', '2'))
TASK_MAX_RETRIES = int(os.getenv('TASK_MAX_RETRIES', '3'))
TASK_TIMEOUT_S = int(os.getenv('TASK_TIMEOUT_S', '300'))  # 5 minutes
```

When you switch from WiFi to wired Ethernet: set HEARTBEAT_TIMEOUT_S=15 as an environment variable. No code changes needed anywhere else. This is what 'production ready' configuration looks like.

9 · Weekly Deliverables — What Gets Done When

Concrete outputs at the end of each week with production quality criteria

WEEK 1

Foundation — Node Registration, Heartbeat, Registry

End-of-week milestone: *Kill any worker process → master detects it dead in ≤20 seconds. Restart master → Redis restores full node registry instantly. Both engineers can explain every line of Week 1 code.*

Component	What Gets Built	Production Outcome
Shared Pydantic models	NodeInfo, HeartbeatPayload, TaskRequest, TaskResult, FileMetadata defined in shared/models.py	*Zero raw dicts anywhere. All inter-node data validated at boundary.*
Node Registry	In-memory dict + Redis backup on master. POST /nodes/register, GET /nodes, GET /nodes/{id}	*Master restart does not lose registry. Redis HSET with TTL.*
Heartbeat system	Workers POST every 5s with CPU%, RAM%, active_tasks. master/registry/health_monitor.py background loop	*Dead detection ≤20s on WiFi. No false positives from WiFi jitter.*
Structured logger	shared/logger.py outputs JSON with timestamp, level, node_id, component, event	*All print() replaced. Every log line parseable by machine.*
Week 1 test suite	tests/test_week1.py — registration, heartbeat, dead detection, Redis restore	*100% pass on actual hardware with all 4 laptops.*
infra/README.md	Step-by-step setup for a stranger to run Week 1 from scratch	*A new engineer follows it and reaches GET /nodes showing all workers.*

WEEK 2

Compute Engine — Scheduling, Docker Execution, Fault Handling

End-of-week milestone: *POST a task → it runs on the correct worker in a Docker container → result retrievable via GET /tasks/{id}. Kill a worker mid-task → task reaper reschedules it. 10 concurrent tasks distribute correctly.*

Component	What Gets Built	Production Outcome
Task database	PostgreSQL tables for tasks. Async SQLAlchemy ORM. TaskStatus enum state machine.	*Async DB — no blocking the event loop. Full task history queryable.*
Resource-aware scheduler	Scoring formula in master/scheduler/scheduler.py. Unit tested with mock nodes.	*Always picks least-loaded worker. Never sends to dead node.*
Task API	POST /tasks, GET /tasks/{id}, GET /tasks, DELETE /tasks/{id} on master	*Returns immediately (async). Client polls for result. No blocking.*
Docker executor	worker/executor/docker_runner.py — docker-py SDK with all security params	*CPU/RAM limits enforced. network=none. Container auto-deleted.*
Concurrency control	asyncio.Semaphore(3) per worker. asyncio.Queue for internal queuing. 429 handling on master.	*No worker can be overwhelmed. Overflow gracefully redirected.*
Task reaper	Background loop every 30s finds orphaned RUNNING tasks on dead workers → FAILED → retry	*No task stuck in RUNNING forever. Max 3 retries before permanent FAILED.*

Week 2 test suite	tests/test_week2_integration.py — full lifecycle, bad image, concurrent tasks, worker death	*100% pass. Includes deliberate worker kill mid-execution.*

WEEK 3

Storage + Functions — Replication, Recovery, Lambda Execution

End-of-week milestone: *Upload a file, kill a worker that holds chunks → download still works from replica. Upload a Python function, invoke it with a payload → correct output returned in ≤5 seconds. Re-replication completes within 60 seconds of failure.*

Component	What Gets Built	Production Outcome
Storage models + repo	FileMetadata, FileChunk Pydantic + ORM models. storage_repository.py with full CRUD.	*Full metadata in PostgreSQL. Chunk locations always queryable.*
Upload pipeline	master/storage/storage_manager.py — chunk, pick 2 workers, POST, verify MD5, save records	*MD5 verified on every chunk. Partial upload rolled back on failure.*
Chunk storage on workers	worker/storage/local_storage.py — POST /chunks, GET /chunks/{id}, checksum API	*Checksums recomputed on retrieval. Mismatch detected before returning.*
Replication monitor	Background 60s loop detects under-replicated chunks, re-replicates from surviving replica	*File status: available / degraded / lost. Never silently lost.*
Lambda function store	Functions stored as text in PostgreSQL. POST /functions, GET /functions/{id}	*Function code versioned by function_id. Never overwritten.*
Lambda executor	worker/functions/function_runner.py — temp file, Docker mount, capture stdout/stderr	*--network=none enforced. Timeout at 30s. Temp file always cleaned up.*
Week 3 test suite	tests/test_storage.py + tests/test_lambda.py — corruption, timeout, concurrent invocations	*100% pass. Includes deliberate chunk corruption test.*

WEEK 4

Monitoring, Hardening, Documentation

End-of-week milestone: *Grafana shows all 4 nodes in real time. 50-task stress test completes with zero stuck tasks. A stranger clones the repo, follows the README, and has the full system running in under 30 minutes.*

Component	What Gets Built	Production Outcome
Prometheus metrics	/metrics endpoint on all nodes. Prometheus scrape config for all 4 targets. 10 key metrics exposed.	*All targets green in Prometheus UI. No scrape errors.*
Grafana dashboards	4 dashboards: cluster overview, worker nodes, task history, storage health. Exported as JSON to repo.	*Dashboards importable from repo JSON. Real-time updates at 15s refresh.*
Log aggregation	POST /logs on master. Workers fire-and-forget JSON logs. GET /logs queryable by node/level/task_id.	*Any task fully traceable across master and worker logs by task_id alone.*
Stress test suite	tests/test_stress.py — 50 concurrent tasks, worker deaths, replication failures, master restart	*All 50 tasks reach terminal state (COMPLETED or FAILED). Zero stuck.*
Redis Streams queue	Replace direct HTTP dispatch with Redis Streams producer/consumer for durable task delivery	*Unacked tasks auto-requeued if worker dies between receive and ACK.*

README + ARCHITECTURE.md	Full setup guide + decision records for every major technical choice	*New engineer runs system from scratch in ≤30 min. Every decision explained.*
Git tag v1.0.0	Tag main branch after all tests pass and documentation complete	*Reproducible release. Anyone can checkout v1.0.0 and run it.*

10 · Success Criteria — How We Know It Works

Concrete, testable, unambiguous definitions of done

Functional Success Criteria

#	Criterion	How to Test It	Pass / Fail
1	Task submitted to master runs on a worker and returns output	POST task with image=alpine, command=echo hello. GET result shows output='hello\n'	Pass if output matches
2	Scheduler never sends tasks to dead workers	Kill worker-1. Submit 20 tasks. GET /nodes — none of the 20 tasks have worker_id=worker-1	Pass if zero assignments to dead worker
3	File downloadable after the storing worker dies	Upload file. Note which workers hold chunks. Kill one. Download — must succeed.	Pass if download returns correct bytes
4	Re-replication happens within 60 seconds	After step 3, wait 60s. Check GET /files/{id} — status returns to 'available'	Pass if status = available within 60s
5	Lambda returns correct output	Upload function that adds x+y. Invoke with {x:5,y:3}. Result must show {result:8}	Pass if output matches
6	Master restart does not lose node registry	Kill master. Restart. Immediately GET /nodes — all alive workers appear.	Pass if workers appear without re-registering
7	50 concurrent tasks all reach terminal state	Run tests/test_stress.py. All 50 tasks reach COMPLETED or FAILED within 10 minutes.	Pass if zero tasks stuck in RUNNING after 10 min
8	Dead node detected in ≤20 seconds	Kill worker process. Watch GET /nodes — status must flip to dead within 20s.	Pass if flip happens ≤20s

Production Quality Criteria

#	Criterion	Why It Matters
1	Every inter-node call uses Pydantic models — no raw dicts	Guarantees data shape at every boundary. Type errors caught before they become runtime bugs.
2	All configuration in infra/config.py with env var overrides	WiFi → wired switch requires zero code changes. New environments require zero code changes.
3	Every public function has a type signature and docstring	Code is self-documenting. Future engineer can understand any function without reading its callers.
4	No task can stay RUNNING indefinitely	Task reaper + container timeout guarantee every task reaches a terminal state. No stuck states.
5	Prometheus shows all 4 targets green with no scrape errors	Monitoring works. If a node dies, the gap in metrics is itself an alert.
6	Any task traceable end-to-end using task_id alone	Grep logs by task_id and reconstruct the full story: submitted, scheduled, dispatched, executed, returned.
7	README allows a stranger to run the full system in ≤30 minutes	System is operable by someone not on the team. This is the definition of production documentation.

#	Criterion	Why It Matters
8	All tests pass on actual multi-laptop hardware, not just local mocks	Tests that only pass on localhost are not integration tests. Real WiFi, real Docker, real latency.

The Final Gauge

Sit down in front of the running system with someone who has never seen it before. In 15 minutes, can you explain what every component does, why every design decision was made, and what happens when any one component fails? If yes — you have built something production-ready. If there are components you cannot explain or failure modes you have not considered — that is your remaining work.